

PRODUCTSTAR

АРХИТЕКТУРА И СЕРВЕРНАЯ ЧАСТЬ



Содержание урока

- ✦ Общее понимание работы сайтов и обмена данными
- ✦ Различие между бэкендом и фронтендом
- ✦ Архитектура веб-сервисов
- ✦ Качество кода и инструменты поддержки
- ✦ Воркшоп



Senior Python Developer

LinearBee

Inteletravel

Lew & Dowski Capital, LLC

АРТЁМ САМОЙЛОВ

- ✦ Senior Python разработчик с 4+ годами опыта в веб-разработке
- ✦ Специализируюсь на backend разработке, архитектуре и CI/CD
- ✦ Работаю с Django, FastAPI, AWS и другими современными технологиями
- ✦ Увлечен AI, автоматизацией и постоянным обучением

ОБЩЕЕ ПОНИМАНИЕ РАБОТЫ САЙТОВ И ОБМЕНА ДАННЫМИ

Клиент-сервер: диалог в сети



Ваш браузер (клиент) запрашивает информацию у сервера. Сервер обрабатывает запрос и отправляет ответ обратно клиенту

Веб-сервер: сердце сайта

Веб-сервер — это программа, которая «живет» на сервере и управляет HTTP-запросами. Он принимает запросы от клиентов, обрабатывает их и отправляет ответы

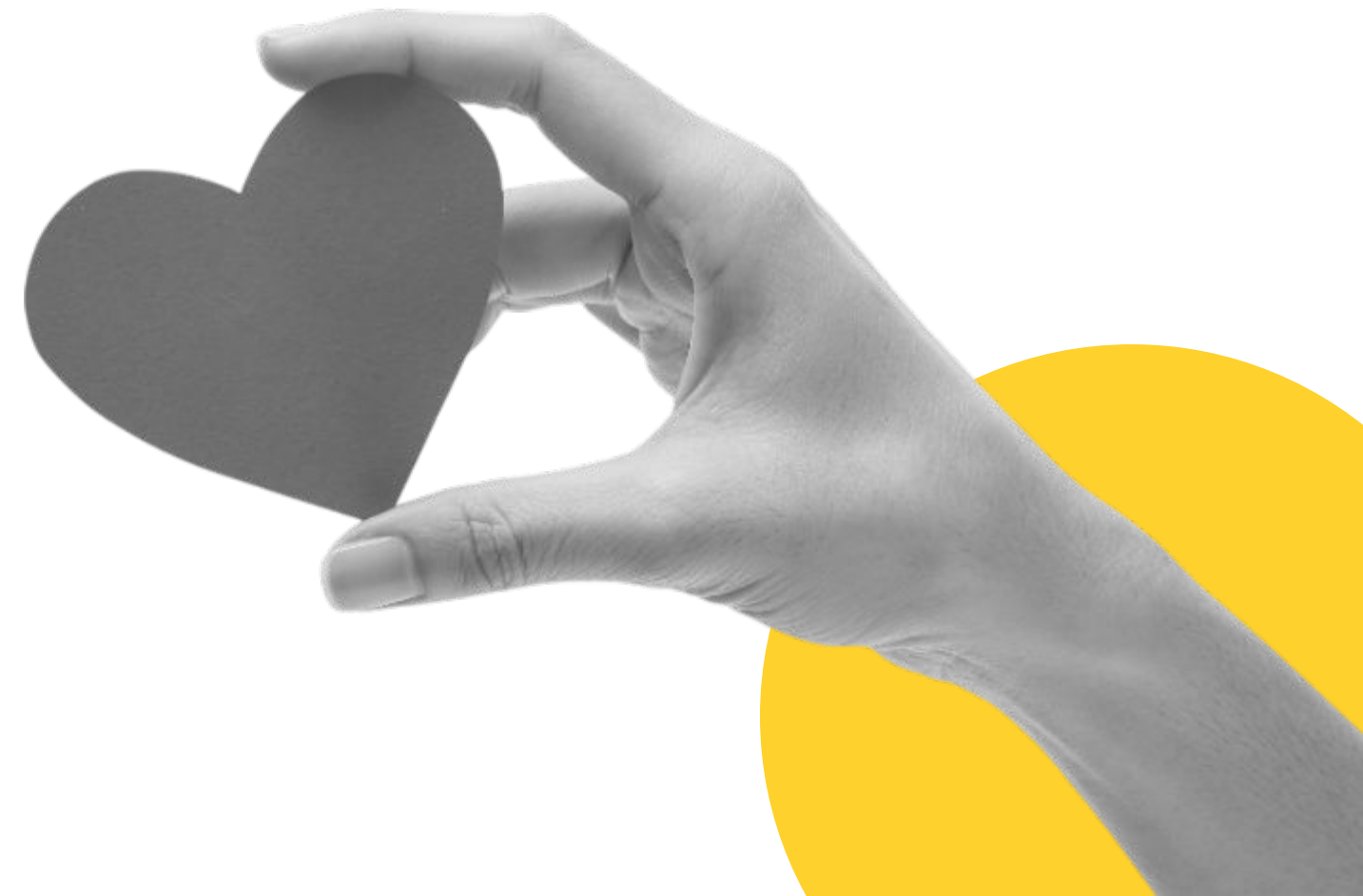
Примеры



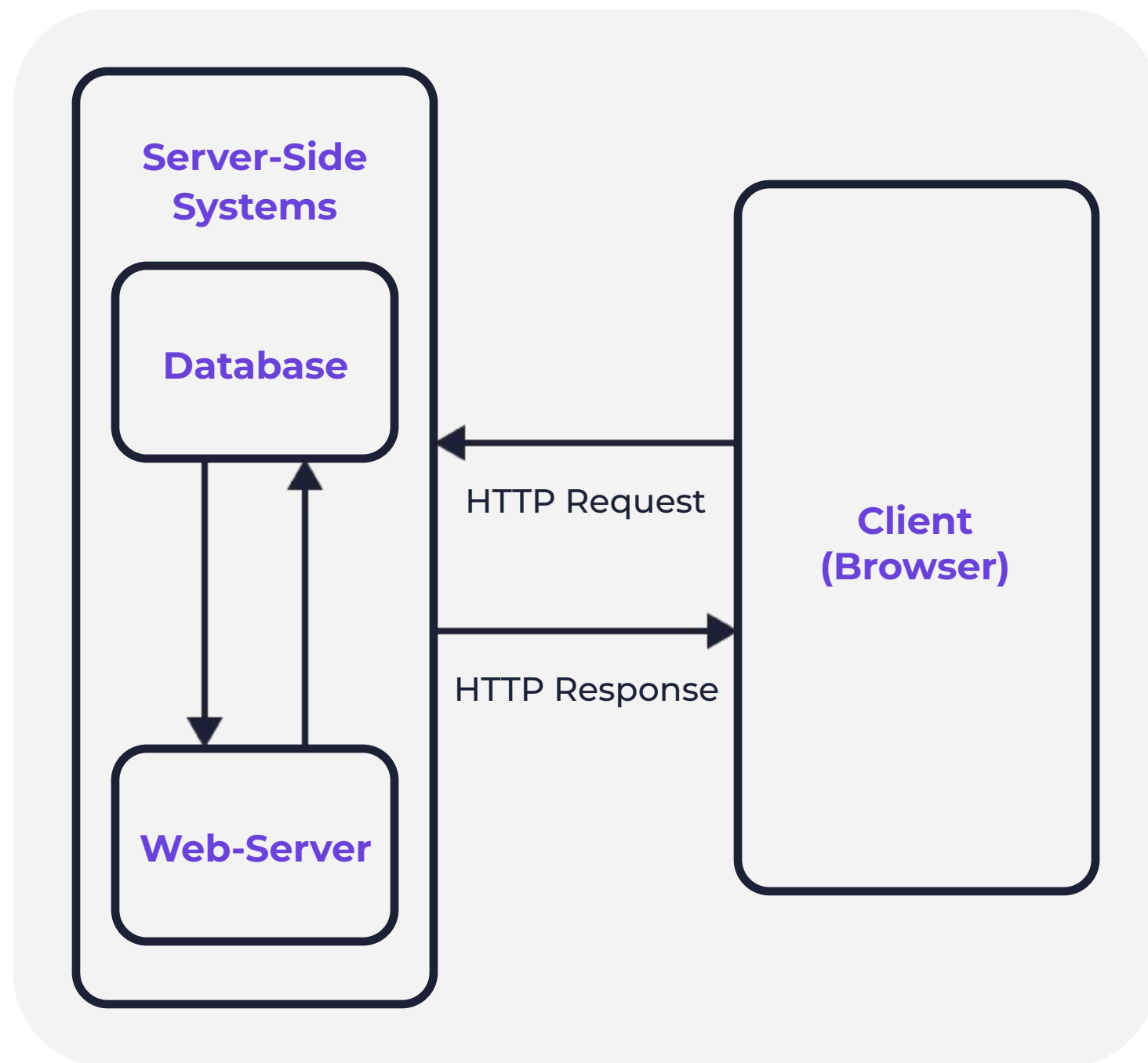
Apache



Nginx



HTTP: язык общения клиента и сервера



HTTP (HyperText Transfer Protocol) — это протокол, используемый для обмена данными между клиентом и сервером. Он работает как общий язык, который понимают обе стороны. Каждый запрос и ответ имеют строго определенную структуру

HTTP методы, коды и тело запроса

Основные методы HTTP

GET

Получение данных. Данные обычно передаются в URL (в параметрах запроса)

POST

Отправка данных. Данные передаются в теле запроса. Например, информация из формы

PUT

Обновление данных. Данные для обновления передаются в теле запроса

DELETE

Удаление данных

Коды состояния (примеры)

- 200 OK
- 404 Not Found
- 500 Internal Server Error

Тело запроса

Содержит данные, которые отправляются на сервер. Например, при заполнении формы, в теле запроса будут значения полей формы



Что возвращает сервер

Тело ответа содержит данные, которые сервер возвращает клиенту. Это может быть:

HTML

Код для отображения веб-страницы

JSON

Данные в формате JSON, часто используется для API

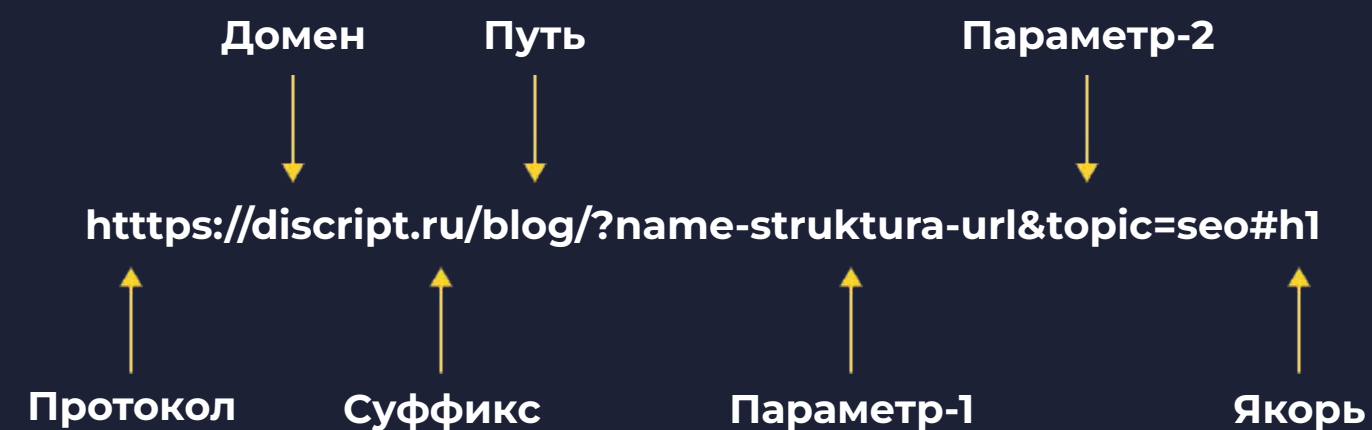
**Изображения,
видео, другие
файлы**

Различные типы мультимедиа

Тип данных указывается в заголовках ответа



URL: адрес ресурса в интернете



`https` — протокол
`discript` — домен
`ru` — суффикс
`blog` — путь
`?name-struktura-url` — параметр-1
`&topic=seo` — параметр-2
`h1` — якорь

URL (Uniform Resource Locator) — это адрес ресурса в интернете. Он указывает, где находится нужная информация

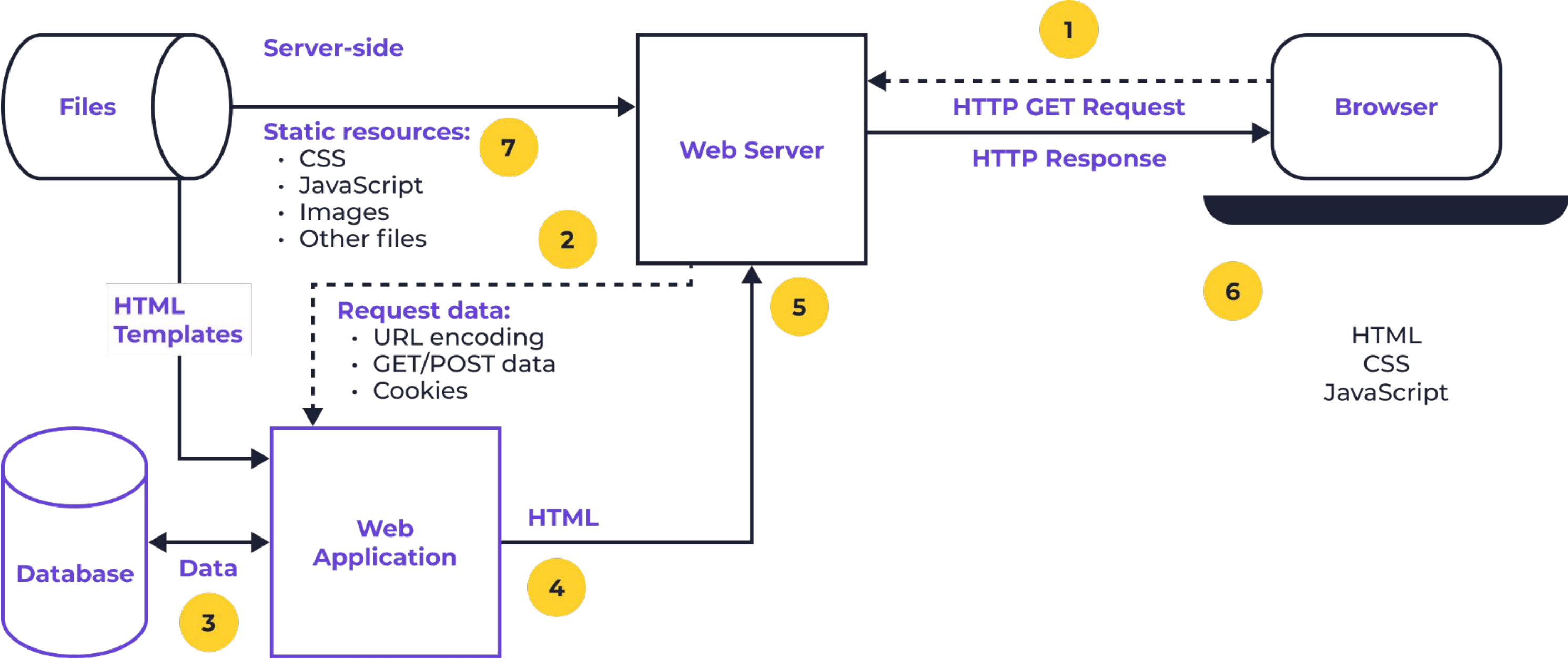
Как URL помогает найти нужную информацию

URL позволяет браузеру найти нужный сервер и конкретный ресурс на этом сервере. Каждая часть URL имеет свое значение и помогает точно определить местоположение информации

Например, `/path/to/resource` указывает на конкретный файл или страницу на сервере



Как это работает вместе



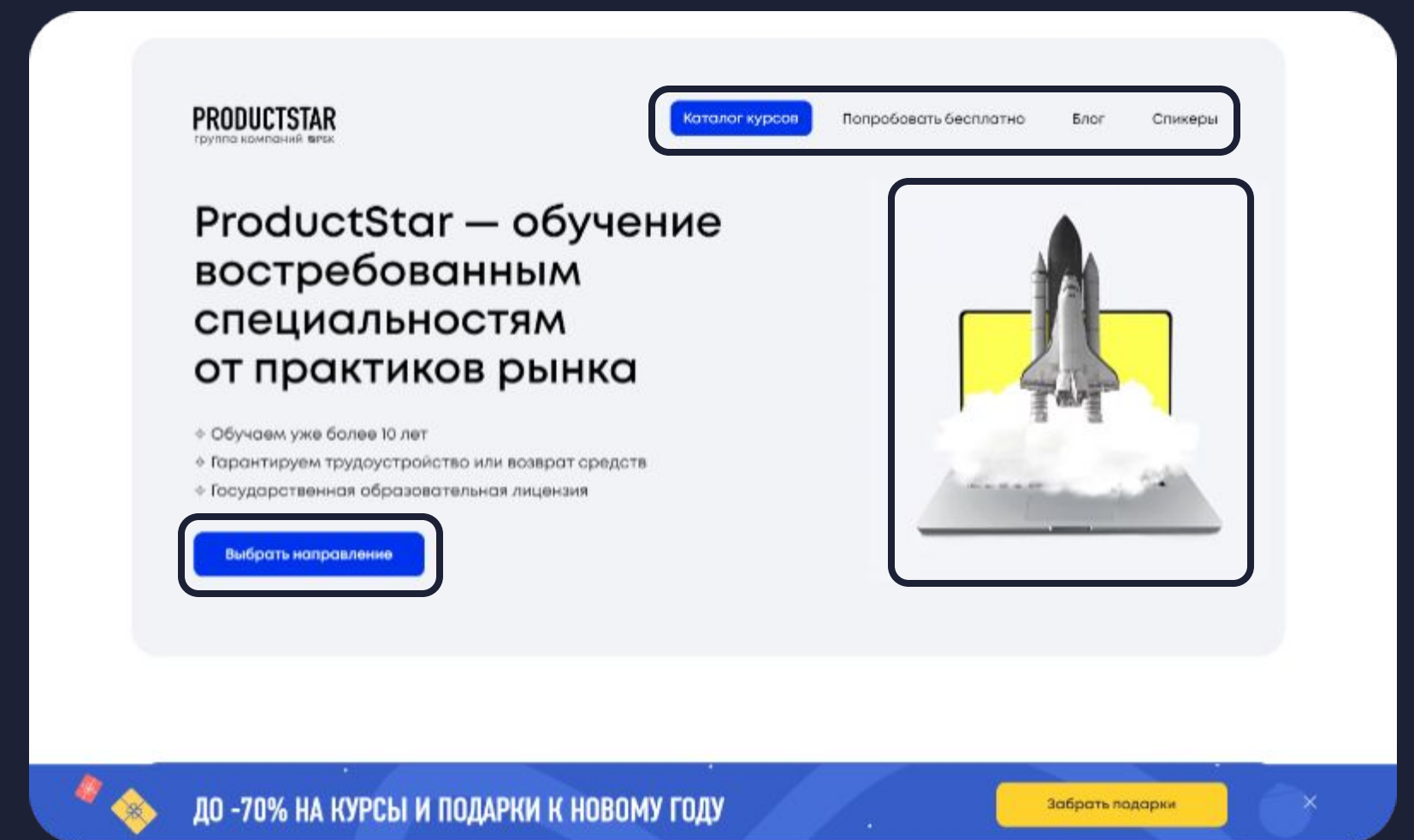
РАЗЛИЧИЕ МЕЖДУ БЭКЕНДОМ И ФРОНТЕНДОМ

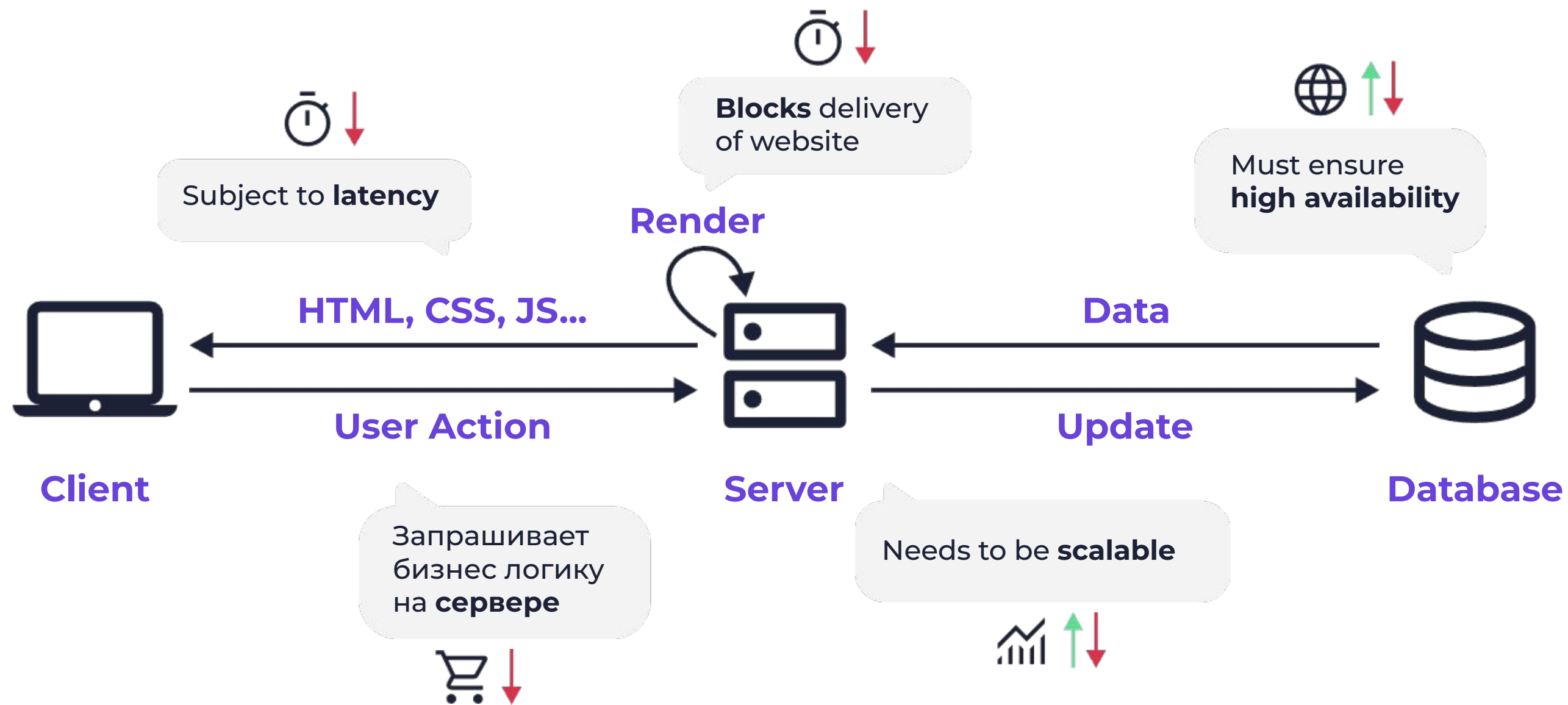
Фронтенд: витрина сайта

Фронтенд — это все, что вы видите и с чем взаимодействуете на сайте. Это как витрина магазина. Фронтенд отвечает за отображение информации, обработку действий пользователя (клики, ввод текста) и взаимодействие с бэкендом

Основные инструменты

HTML, CSS, JavaScript



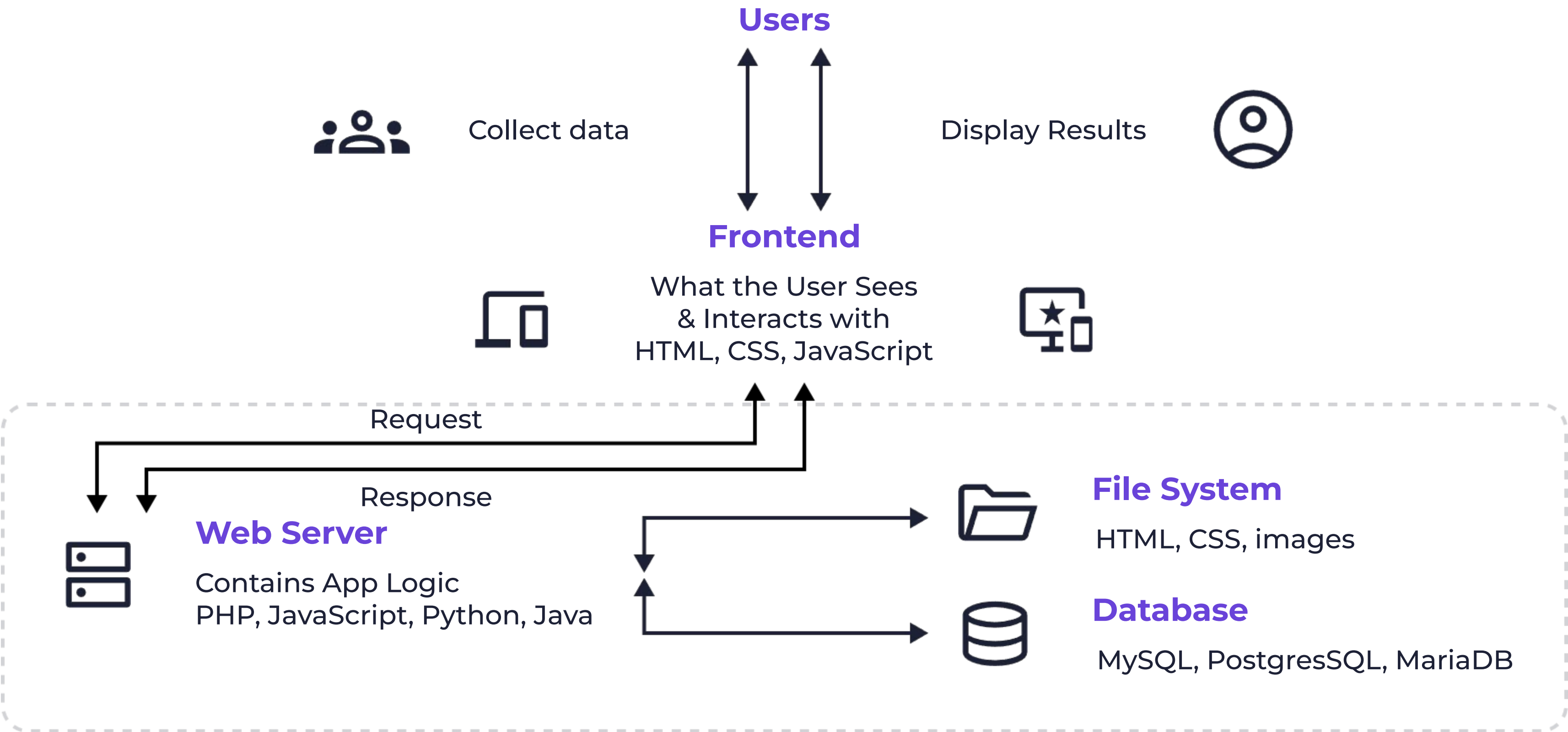


Бэкенд: мозг и сердце сайта

Бэкенд — это «мозг» сайта, который работает за кулисами. Он обрабатывает данные, выполняет бизнес-логику (например, регистрация пользователя, обработка заказа), взаимодействует с базой данных

Python с фреймворком Django — популярные инструменты для создания бэкенда

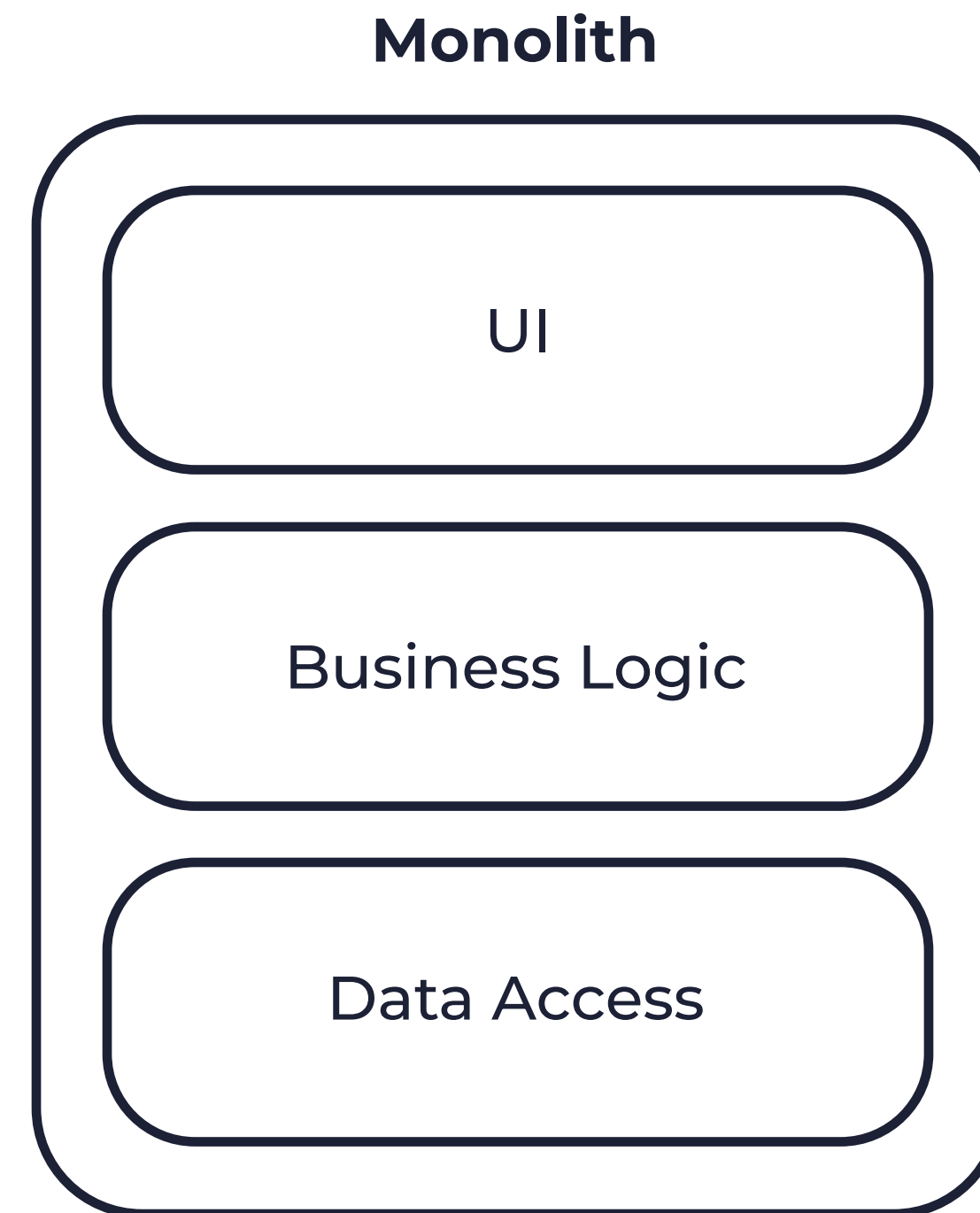
Фронтенд + бэкенд = работающий сайт



АРХИТЕКТУРА ВЕБ-СЕРВИСОВ

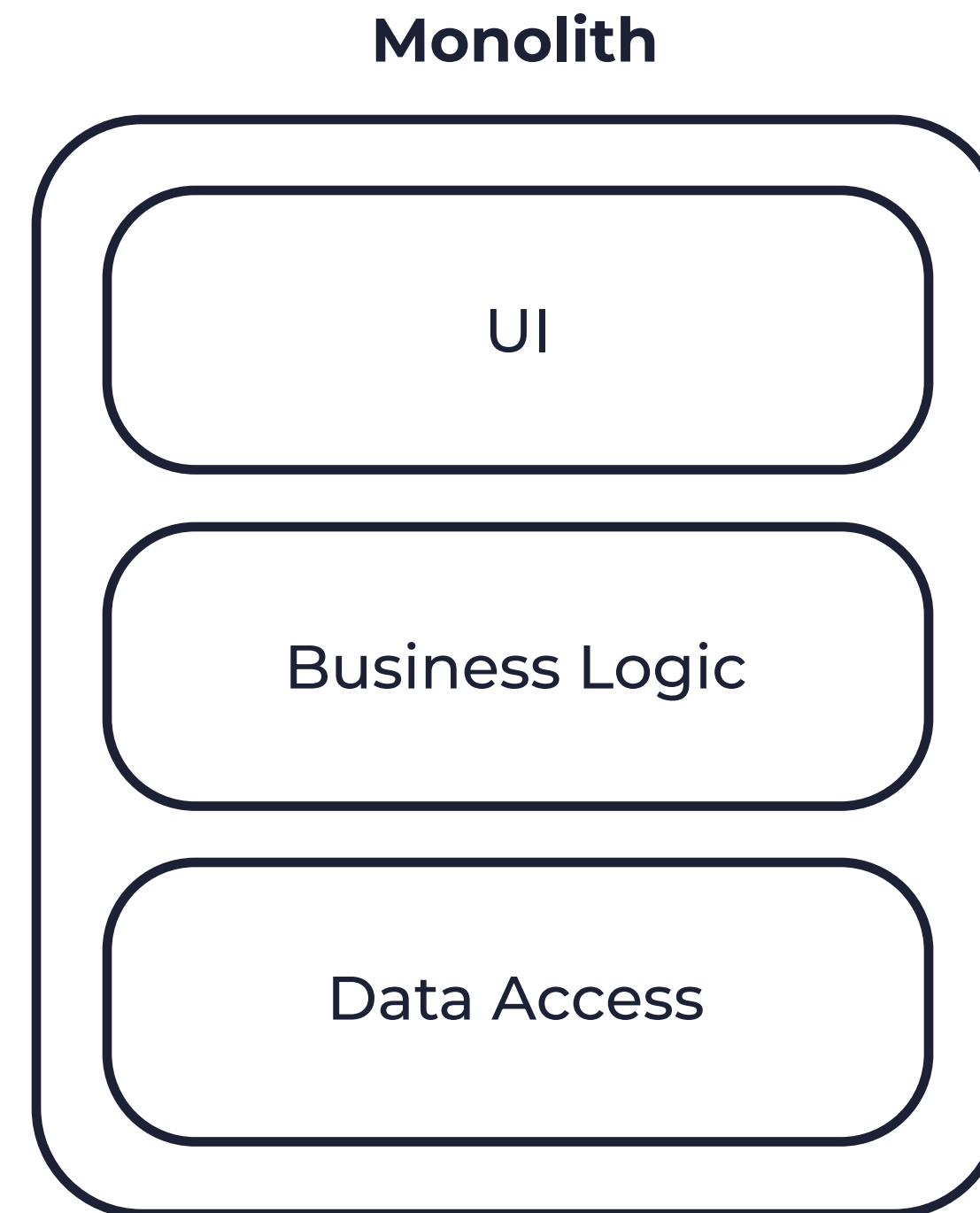
Монолитная архитектура: простота и цельность

- + Простота разработки и развертывания
- + Производительность
- + Легкое тестирование
- + Упрощенное управление
- + Легче для понимания



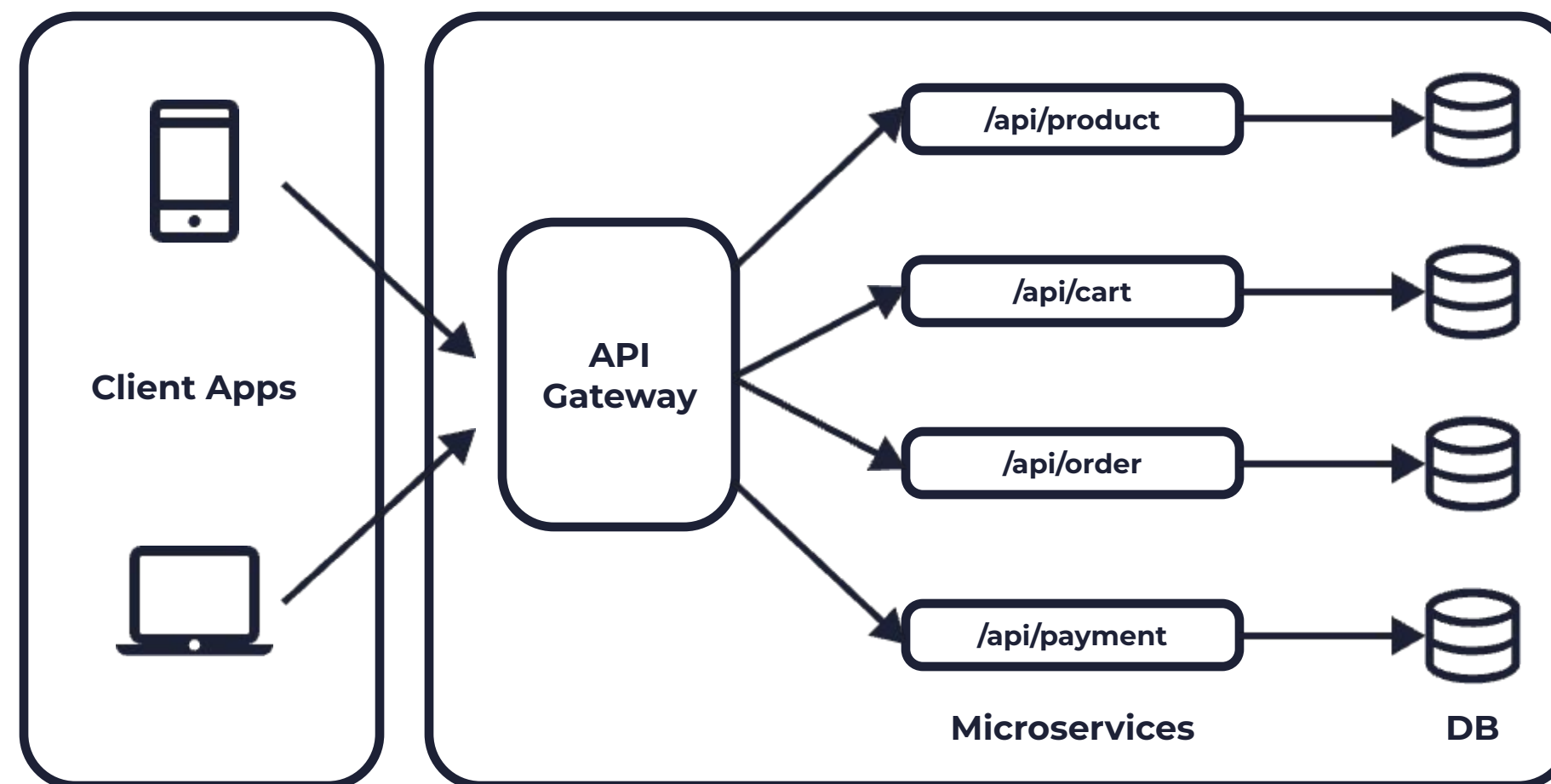
Монолитная архитектура: простота и цельность

- Сложно масштабировать
- Изменения затрагивают весь проект
- Низкая отказоустойчивость
- Технологический застой
- Большая кодовая база



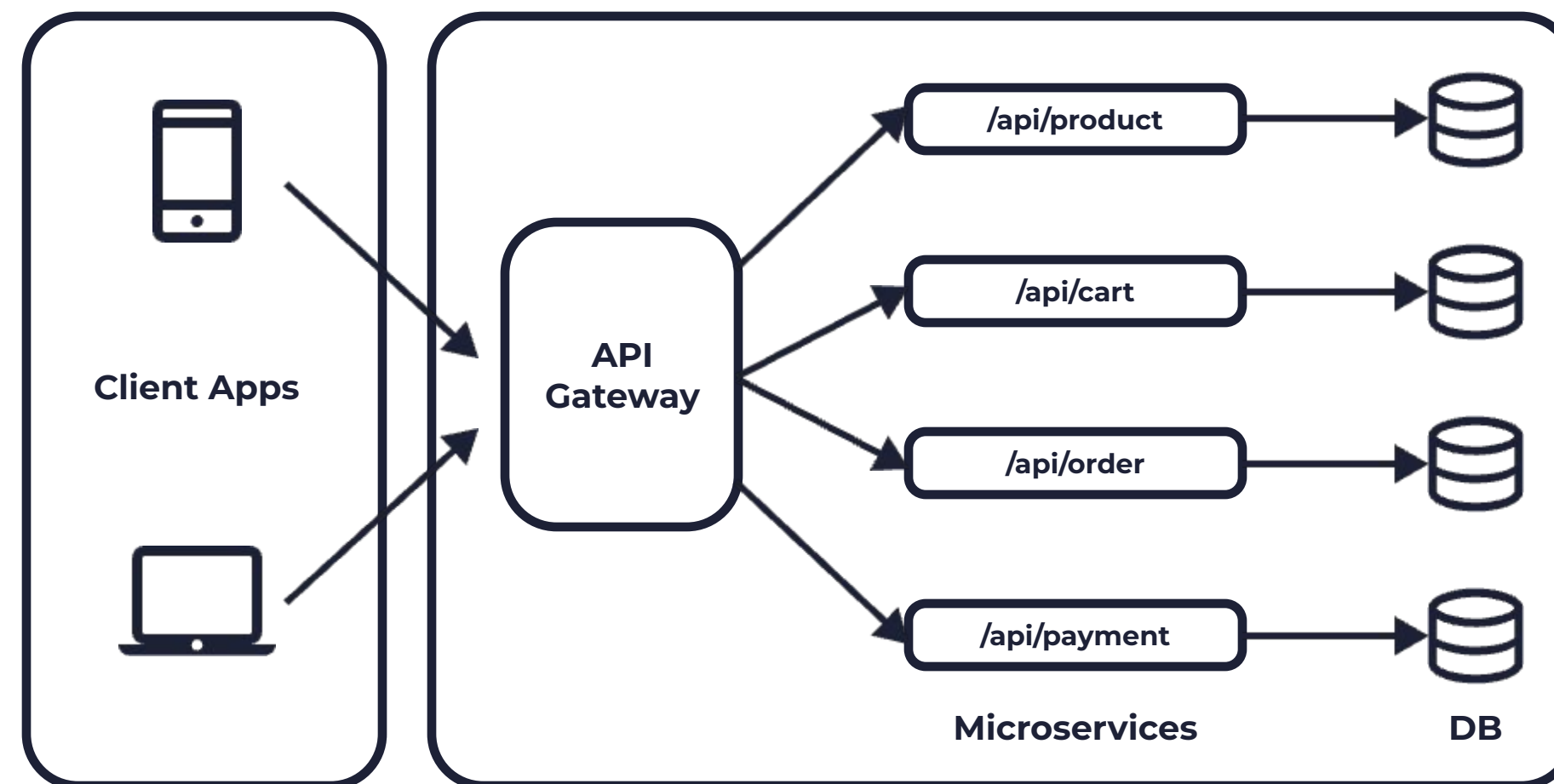
Микросервисная архитектура: гибкость и масштабируемость

- + Масштабируемость
- + Отказоустойчивость
- + Гибкость
- + Быстрое развертывание



Микросервисная архитектура: гибкость и масштабируемость

- Сложность разработки и развертывания
- Сложная отладка
- Потенциальные проблемы с производительностью из-за сетевого взаимодействия



Выбор правильного инструмента

Выбор архитектуры зависит от конкретных требований проекта

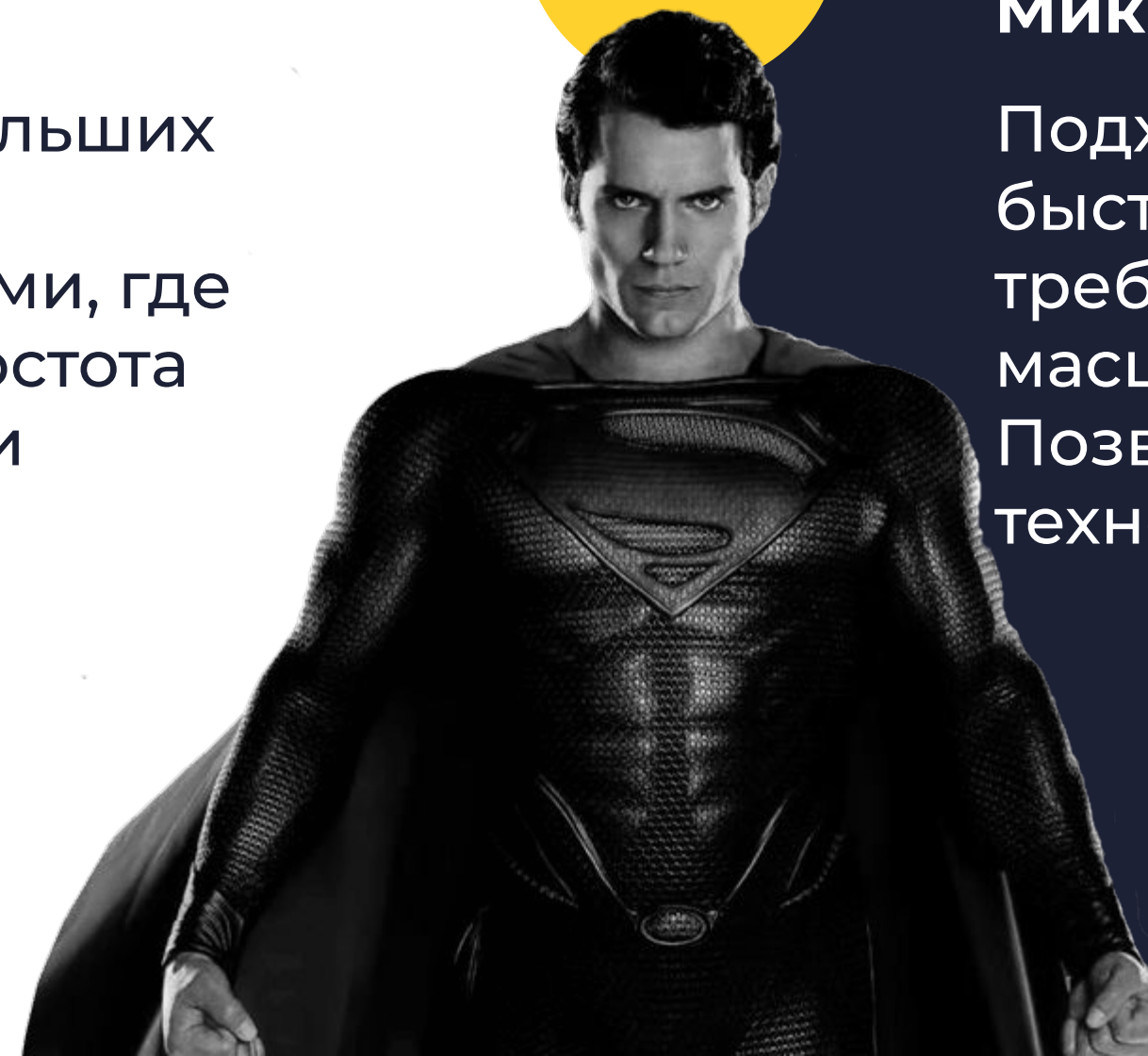
VS

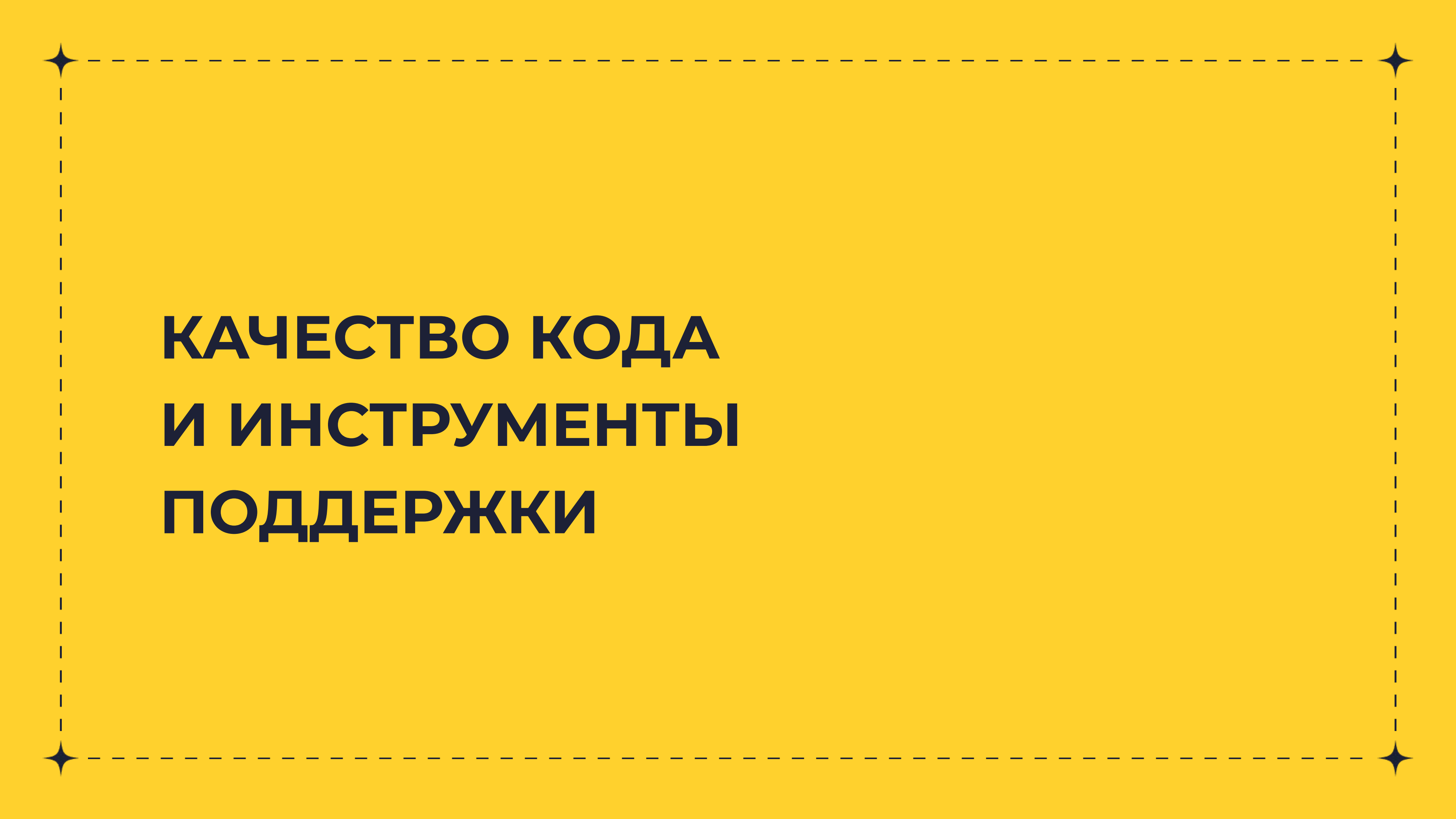
Монолит

Отличный выбор для небольших проектов, MVP, проектов с ограниченными ресурсами, где скорость разработки и простота важнее масштабируемости

Микросервисы

Подходят для сложных, быстрорастущих проектов, требующих высокой надежности, масштабируемости и гибкости. Позволяют использовать разные технологии для разных сервисов





КАЧЕСТВО КОДА И ИНСТРУМЕНТЫ ПОДДЕРЖКИ

Чистый код: инвестиция в будущее проекта

- Читаемый код — понятный код
- Поддерживаемый код — легко изменяемый и расширяемый код
- Чистый код снижает количество ошибок и упрощает отладку
- Улучшает командную работу
- Долгосрочная экономия времени и ресурсов




```

1 import sys,os,random,datetime
2
3 class UglyClass:
4     def __init__(self,x,y,z):
5         self.X=x;self.Y=y;self.Z=z
6
7     def ugly_function(x,y):
8         if(x>0):
9             if(y>0):
10                 return True;
11             else:
12                 return False
13         elif(x<0):return None;
14 l = [1,2, 3, 4,5]
15 d={'a':1,'b':2,
16   'c':3,'d': 4}
17 class TooManySpaces ( object ):
18     def __init__( self ):
19         self.var1=1
20         self.var2 =2
21         self.var3 =3
22 x=[i for i in range(1,1000) if i%2==0 if i%3==0 if i%4==0 if i%5==0]

```

Линтеры: автоматическое выявление проблем

Линтеры (pylint, flake8, ruff) анализируют код на наличие ошибок, потенциальных проблем и несоответствий PEP 8

```
1 class UglyClass:
2     def __init__(self, x, y, z):
3         self.X = x
4         self.Y = y
5         self.Z = z
6
7
8 def ugly_function(x, y):
9     if x > 0:
10         if y > 0:
11             return True
12         else:
13             return False
14     elif x < 0:
15         return None
16
17
18 l = [1, 2, 3, 4, 5]
19 d = {'a': 1, 'b': 2, 'c': 3, 'd': 4}
20
21
22 class TooManySpaces(object):
23     def __init__(self):
24         self.var1 = 1
25         self.var2 = 2
26         self.var3 = 3
27
28
29 x = [i for i in range(1, 1000) if i % 2 == 0 if i % 3 == 0 if i % 4 == 0 if i % 5 == 0]
```

Форматтеры: автоматическое форматирование кода

Скриншот примера работы форматтера, показывающий преобразование кода

Форматтеры (black, ruff) автоматически приводят код к единому стилю, согласно PEP 8



ВОРКШОП

Содержание воркшопа

- ✦ Научимся настраивать ruff в `pyproject.toml`
- ✦ Разберем основные возможности
- ✦ Добавим автоматизацию — pre-commit hook



**Переходим в PyChart
для демонстрации!**

ДОМАШНЕЕ ЗАДАНИЕ

Домашнее задание

- 1 Изучить исходный код и попытаться самостоятельно найти проблемы в коде без помощи линтера
- 2 Изучить документацию ruff и провести его базовую настройку
- 3 Применить ruff к исходному коду проекта, посмотреть на результат и донастроить ruff, если это необходимо



СПАСИБО ЗА ВНИМАНИЕ!



samoilovartem



samoilovartem



ExpatDevDiary